SOURCE-LEVEL GUIDE • KIRODOTDEV/KIROCREW

Kiro Crew 源码导读

Kiro Crew 是运行在你自己硬件上的持久开发工作台：一个 gateway 进程把 CLI、Dashboard、消息渠道接到 ACP agent。这份导读讲它的主干、安全模型、记忆和编排，以及在 220 万行 Python 里该从哪读起。

kirodotdev/KiroCrew @ 083e552

怎么读这份导读

这份导读写给想读懂、贡献或定制 Kiro Crew 的工程师。这个仓库非常大：`src/kiro_crew` 有 4245 个 Python 文件，约 223 万行（含测试夹具和 vendored 代码），前端还有 7000 多个文件。所以这里**只讲主干和边界**，不逐模块介绍。

读完你应该能做到：说清一条消息怎么走到模型再回来；知道安全保证到底落在哪一层；知道一个改动该先读哪份规格；判断该写 App、Skill、MCP 工具，还是改核心。

全文顺序：

1. **全景**：三层结构、目标与限制。
2. **Gateway 与会话**：进程模型、session key、生命周期。
3. **ACP 与 harness**：唯一的 provider，多个后端。
4. **上下文与记忆**：提示怎么拼，记忆怎么沉淀。
5. **安全模型**：治理、闸门、沙箱、审计，这是项目的第一原则。
6. **编排**：子 agent、Cron、TaskRunner、心跳。
7. **App 边界**：核心和 App 怎么分。
8. **端到端**：一条 Dashboard 消息。
9. **许可证、贡献与定制、源码索引**。

> **要点 · 固定版本**
>
> `kirodotdev/KiroCrew` @ `083e552`（2026-11-06，版本 `0.13.0`）。我没有运行过它。下面的内容来自阅读代码和仓库文档，这个仓库的 `docs/` 非常详细，很多结论直接引用了它们。

全景：问题、目标、限制

三层结构

理解 Kiro Crew 先要分清三层（架构总览）：

层	是什么	负责
ACP harness	agent 的运行时，默认是 `kiro-cli`	LLM 连接、工具执行、MCP、上下文压缩
Agent 配置	`~/.kiro/agents/` 下的 JSON	系统提示、启用的工具、MCP 服务
Kiro Crew	单个 asyncio 进程，即 gateway	把多个入口接到 harness，补上运行时不管的东西

也就是说，Kiro Crew **自己不调用 LLM**。它是 harness 上面的一层：会话管理、记忆、调度、治理。

能力	只用 kiro-cli	加上 Kiro Crew
会话	每个终端一个	频道线程、Dashboard 槽位、Cron、子 agent 同时运行
入口	终端	CLI、Web、桌面、Slack、Discord 等
持久化	按目录的转录	跨会话记忆
调度	无	Cron、TaskRunner
工具闸门	agent 配置	独立的 PreToolUse 闸门 + agent 改不了的治理上限
进程韧性	手动重启	warm pool、断路器、崩溃恢复、孤儿进程跟踪

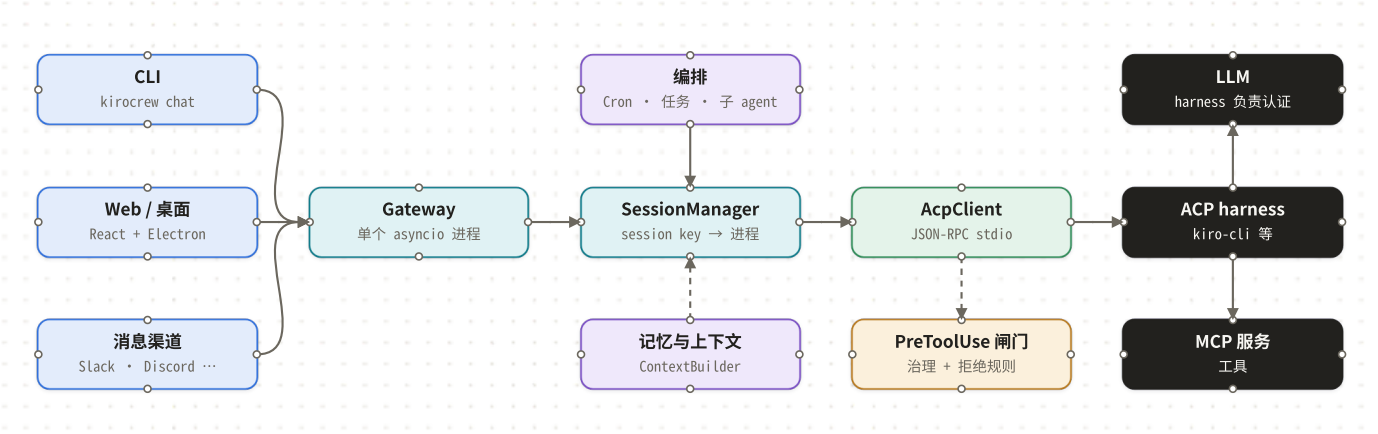
这张表整理自 `overview.md`。

要求 / 目标 / 限制

项目	内容
核心目标	持久、无人值守、会自我改进的开发工作台，运行在用户硬件上
第一原则	Safety first：每个动作都可门控、可审计（`TENETS`，排序有优先级）
功能要求	多入口、跨会话记忆、Cron、自主任务、App 平台
规模压力	单进程里并发很多会话；每个会话一个子进程
外部限制	依赖外部 harness；跨 macOS/Linux/Windows；公开仓库是内部包的去 Amazon 化分支
关键手段	ACP 协议隔离 harness；`POLICY ∩ PROFILE` 治理；OS 沙箱；App 边界；规格文档路由
许可证	Apache-2.0

总图

入口、gateway、harness



深色是外部进程。所有入口和后台任务最终都走 `SessionManager`。Mermaid 版本见 `High-level shape`。

它从哪来

这个仓库是一个内部包“去 Amazon 化”后的公开分支（`AGENTS.md`）。`oss-fork-boundaries` 列了不能带回来的东西，并由 `internal-content-scan` 检查把关。这对贡献者有直接影响：标记列表本身是私有的，你看不到全部规则。

Gateway 与会话

职责

gateway 是一个 aiohttp + asyncio 进程。入口脚本是 `kirocrew`，指向 `_bootstrap.py`，服务组合在 `slack/gateway.py`（文件名有历史原因），Dashboard 在 `dashboard/server.py`。

核心概念：session key

每个会话是一条独立 ACP 连接，用 `session_key` 标识，key 编码了它的来源。不同来源的生命周期不同，这是刻意的：**人盯着的界面保留会话，后台任务不保留**。

调用方	模式
频道	每线程长期存在，`finally` 里 `release()`，靠空闲回收
Dashboard 槽位	每槽位长期存在，用户关闭
Cron	`cron:{id}`；非持久时每次一个新 key，上下文不累积
心跳	一轮共用 `HEARTBEAT_KEY`，轮末回收
子 agent	`subagent:{id}`，会话文件保留给 `spawn_continue`

来自 `Lifecycle by caller`。

`SessionManager` 管理这张表。`get_or_create` 是分配入口。

几个默认值（来自 `overview`）：

配置	默认	含义
`session.pool_size`	0	warm pool 默认关闭
`session.timeout_secs`	3600	空闲回收
`agent.chat_turn_timeout_secs`	14400	单轮上限，不能关闭
`agent.tool_approval_timeout_secs`	600	审批窗口，必须在轮次内到期
断路器	连续 5 次失败	强制重置
`session.autocompact_pct`	70%	自动压缩阈值

审批窗口的设计值得看：它要比轮次上限早到期，否则没回答的审批会被报告成轮次超时，真正的原因就丢了。

代码组织：把一个类拆成很多文件

打开 `acp/client_turn.py` 会看到这样的文件头：函数第一个参数是 `self`，由类按名字绑定，并列出读写的实例字段。这个项目用这种方式把几千行的类拆成多个 owner 模块。`session_parts/`、`taskrunner_parts/`、`agent_parts/` 都是这个模式。

> **注意 · 容易误解的地方**
>
> - **很多文件是 shim。** 比如 `acp_tool_gate.py` 只是重新导出，真实现在 `agent_sdk/`。打开文件先看文件头。
> - **模块级全局变量是有意的。** `hooks.py` 开头保留“看起来没用”的 import，因为 owner 模块都从这里读，测试也替换它们。

关闭顺序

`Shutdown order` 明确写着顺序是承重的。一个例子：杀掉所有子进程会产生 `waitpid` 回收风暴，可能卡住事件循环超过看门狗阈值，所以要**先关看门狗**，否则干净退出会变成崩溃退出。

ACP 与 harness

职责

Kiro Crew 只有一个 LLM provider，就是 ACP。`agent.provider` 固定为 `acp`，后端由 `agent.acp_backend` 选（`overview`）。AGENTS 的开头补充说明：未选择时默认跑 KAS 引擎（`AGENTS.md`）。这两处的措辞不完全一致：overview 说 `kiro-cli` 是默认，AGENTS 说走 KAS，通过 `kiro-cli acp --agent-engine v3` 或自带的引擎。以代码 `kas_route.py` 为准。

后端

`backends.py` 是“这个构建能驱动哪些 harness”的唯一判定处：

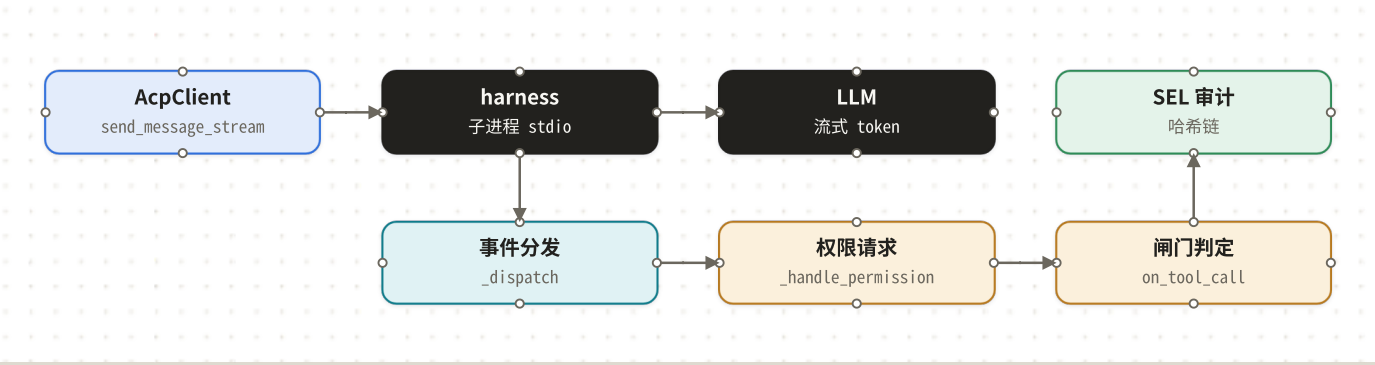
后端	常量
kiro-cli	`ACP_BACKEND_KIRO`（值是空字符串）
KAS	`ACP_BACKEND_KAS`
Claude Code	`ACP_BACKEND_CLAUDE`
Codex、OpenCode、Pi	`CODEX`、`OPENCODE`、`PI`
goose、DeepSeek、Copilot、Strands	`GOOSE`、`DEEPSEEK`、`COPILOT`、`STRANDS`

难点：闸门在每个 harness 上都要成立

每个 harness 的权限模型都不同。如果某个 harness 能绕过 Kiro Crew 的闸门，治理就失效了。所以 `harness-parity` 把这件事定义成不变量，并有 `CI 闸门`。

`client_permissions.py` 里能看到这种防御：`_refuse_identity_drift`、`_tripwire_goose_mode`、`_tripwire_pi_gate`、`_refuse_session_config_drift`。它们检查 harness 有没有偷偷改了权限模式。agent 配置说的不算，要验证运行时实际状态。

一轮 ACP 交互



流式入口是 `send_message_stream`，权限处理是 `_handle_permission`。不同 harness 走闸门的路径不同，比如 KAS 在 `kas_wire.py` 调用；我没有逐个 harness 核实。

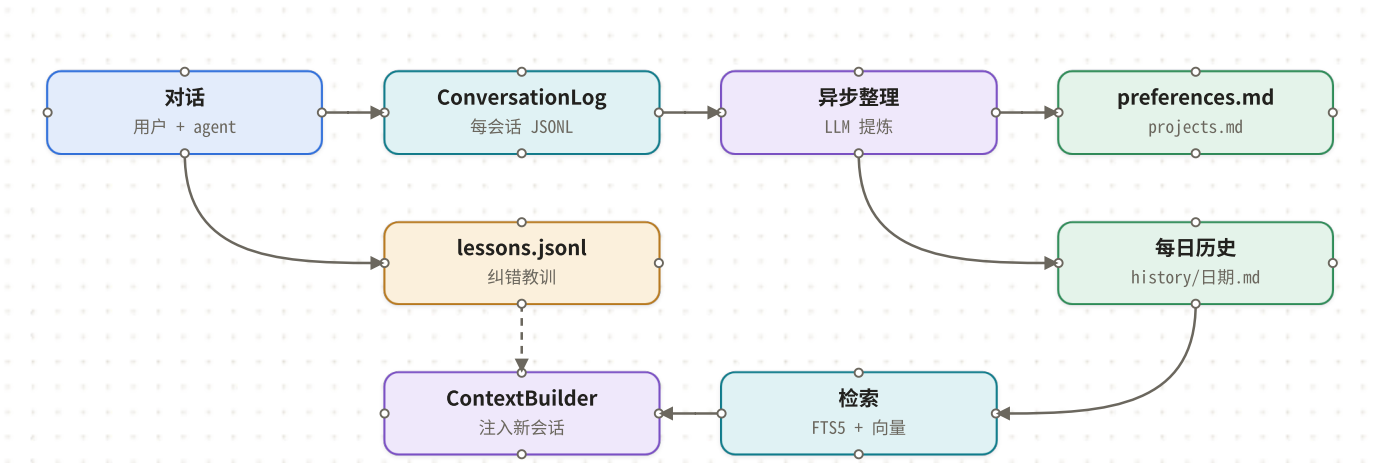
上下文与记忆

职责

`ContextBuilder` 把记忆、技能和 hook 注入的内容拼成一段，在会话第一轮（或上下文重置后）加到用户消息前面。

记忆的生命周期

全局 V1 记忆的流向



对话阈值和空闲阈值见 `Memory lifecycle`：偏好和项目约 30 条消息整理一次，每日历史在 3 小时空闲后整理。存储在 `MemoryStore`。成员绑定的 V2 记忆用 SQLite，不可用时显式失败，不回退到 V1。

记忆会衰减（`Global V1 history decay`）：

距今	注入方式
请求窗口内（默认 14 天）	全文
到第 60 天	摘要：标题加第一条
61–180 天	一行标记
超过 180 天	不读
超过 `history_max_days`（默认 365）	心跳任务删除

embedding 永远在进程内，用 vendored 的 llama-cpp-python，不需要装额外 daemon。

类比 · 记笔记的助理

- **录音** = JSONL 转录，完整但没法直接用。
- **整理笔记** = 异步整理：把录音提炼成 `preferences.md`、`projects.md`。
- **便利贴** = lessons：你纠正过的事，下次直接贴在桌上。
- **翻旧账** = 记忆衰减：上周的看全文，两个月前的看摘要，半年前的只记得有这回事。

助理不会每次都把全部录音重放一遍，这就是“不重放也能受益”。

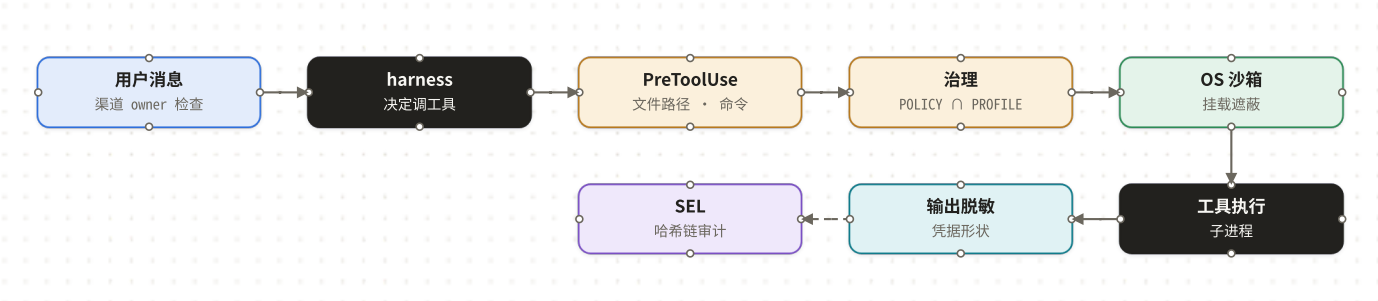
安全模型

这是项目最认真的部分。AGENTS.md 的 `Security invariants` 写得很长，每一条都带着事故编号。

分层

层	作用	位置
Owner lock	消息渠道只接受配置的 owner	渠道层
治理	`POLICY ∩ PROFILE`，最紧的赢	`governance.py`
PreToolUse 闸门	拒绝规则、敏感路径、审批	`HookManager`
OS 沙箱	namespace / Seatbelt 隐藏凭据目录	`sandbox.py`
输出脱敏	擦除凭据形状的文本	`log_redaction.py` 等
SEL	HMAC-SHA256 哈希链审计	`sel.py`

工具调用要穿过的层



分层总结见 `Security layers`，深入内容在 `security-deep-dive.md`。

最关键的一条认识

真正的强制点在 OS 层，不是文本匹配。 闸门可以拦文件工具的路径，但一个 shell 里的 `open()` 根本不经过工具闸门。所以试图用正则堵命令文本是徒劳的：`/./`、重定向、变量、`cd` ……每堵一种写法，只是在无限集合里少了一个。

AGENTS.md 的 `Keystone` 用三种处置描述 crew home 下的每个文件：

处置	含义	例子
`HIDDEN`	所有模式下挂载遮蔽	凭据目录、`.env`
`READONLY`	沙箱内可读不可写	四个治理文件
`VISIBLE`	需要读写，只靠工具闸门	`sel_hmac.key`

反直觉的一点：治理文件是**故意可读**的。遮蔽策略文件会让它回退到宽松的默认值，隐藏上限反而等于移除上限。

> **注意 · 文档之间有不一致**
>
> `overview.md` 说 agent “既不能读也不能写”自己的上限，而 AGENTS.md 明确说可以读、不能写。AGENTS 的说法更新也更详细，并有测试钉住，应以它为准。另外 `sel_hmac.key` 是 `VISIBLE`，AGENTS 自己列为遗留问题：审计密钥没有 OS 层保护。

沙箱分级

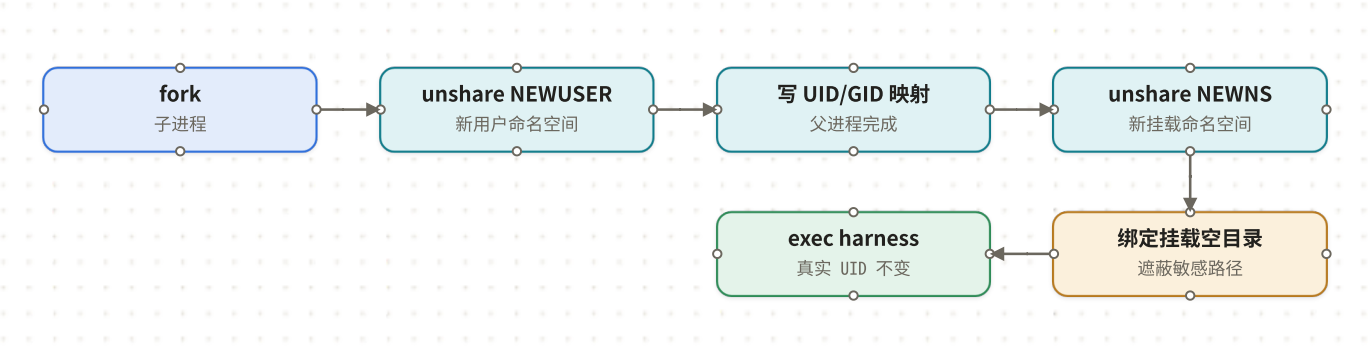
`sandbox.py` 的文件头：

档位	遮蔽	留下
`standard`（`auto` 解析成它）	`~/.gnupg`、`~/.docker`、`~/.azure`、gcloud、crew vault	`~/.aws`、`~/.ssh`、`~/.kube`
`strict`	再加 `~/.aws`、`~/.kube`、`~/.config/gh`、`~/.ssh`	仅 `known_hosts`
`cc`	Claude Code 后端专用	只读的 `~/.aws/config`

默认档位下，agent 里的 shell 能读 `~/.aws` 和 `~/.ssh`。 这是刻意的取舍，让 `aws` CLI 和 git-over-SSH 能用。在意这一点就开 `strict`。另外，OS 机制不可用时沙箱会优雅回退到无沙箱，并记警告（`文件头`）。

Linux 上不需要 root，靠用户命名空间实现：

Linux 沙箱的启动顺序



子进程保留真实 UID，所以工具链正常工作；父进程完全不受影响（`sandbox.py`）。macOS 走 `sandbox-exec` 的 Seatbelt 配置。

治理

`effective = POLICY ∩ PROFILE`。POLICY 是企业上限，启动时从 trust root 加载；PROFILE 是每个界面的收窄。评估器不认识 scope 的名字，只按四种原型工作（`governance.py`）：`ScopedRuleset`、`OrdinalControl`、`CapabilityGate`、`ScopedMap`。所以新增 scope 是改数据，不是改评估器。

文化：不要追拼写

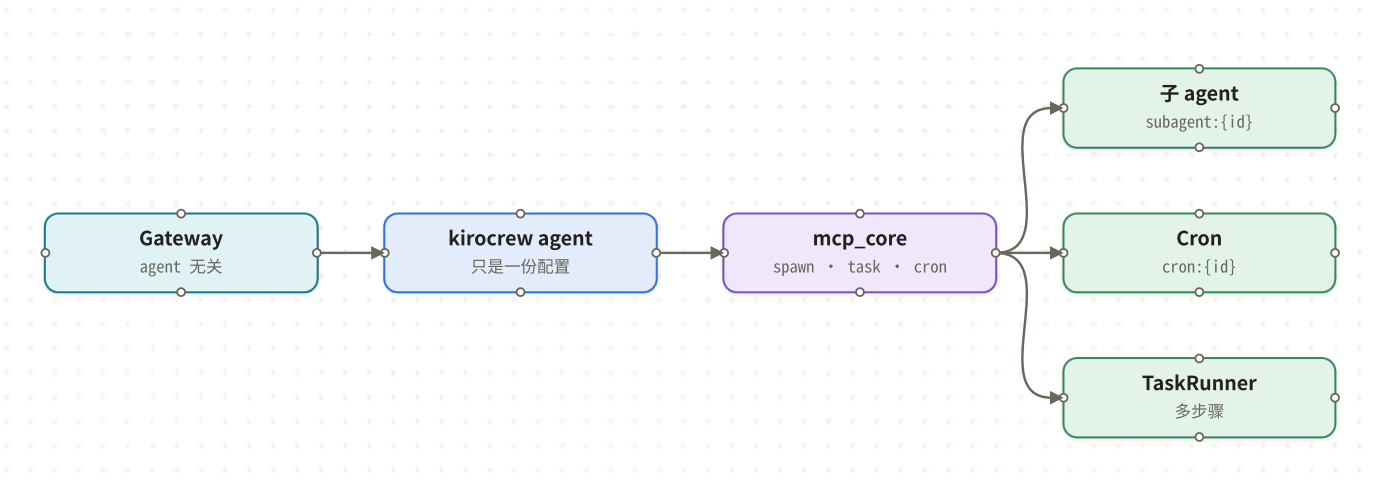
AGENTS 有一条规则叫“正则拼写追逐是 review 异味”，还有一条是“沙箱范围由操作者决定，不由你决定”：加一个没人要求的遮蔽，会在几天后表现为工作莫名停了。#12103 曾在同一段代码上花了 11 轮。贡献安全相关代码前，一定要读这部分。

编排

机制	作用	位置
子 agent	派生隔离的后台 agent	`subagent.py`
Cron	定时任务，CLI 和 gateway 用文件锁共享 `crons.json`	`cron.py`
TaskRunner	规格 → 分解 → 执行 → 重试 → 重规划 → 检查点	`taskrunner.py`
心跳	周期性维护	`heartbeat.py`
动态工作流	工作流引擎	`workflows/`
Crew	agent 团队	`crew-mode`

`kirocrew` 本身只是一个 agent 配置，它之所以是协调者，是因为 gateway 默认用它，并且它持有 `spawn_run`、`cron_add`、`task_run` 这些 MCP 工具（`agent hierarchy`）。**gateway 代码是 agent 无关的，协调行为在配置里。**

协调行为靠 MCP 工具回到 gateway



工具在 `mcp_core.py`。这些后台任务又都通过 `SessionManager` 开新会话，所以和交互会话共用同一套闸门。

Cron 有一条来之不易的规则：脚本正文不能交给 shell 闸门。这条规则试过（#4243），带来一连串误报（#7912、#8563 等），每次都靠再加一层 AST 修补，还是挡不住 `open(a + b)`。现在脚本正文只走整体检测器，运行时靠沙箱（`AGENTS.md`）。

App 边界

第 8 条原则是“一切皆 App”（`TENETS`）。`The app boundary` 把它落成一条线：

留在核心	为什么不能是 App
会话和转录	每个界面读同一份对话
记忆和教训	跨会话共享
审批和 PreToolUse 闸门	价值就在于绕不过去。App 提供的闸门就是带开关的闸门
治理上限	可替换的上限不是上限
事件总线和身份	App 之间的约定基础

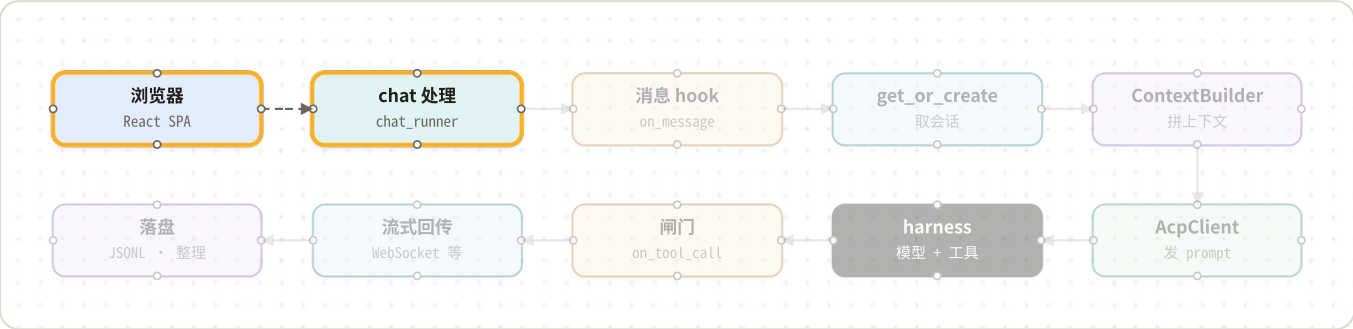
其余都应该是 App。**仓库不再接受新的内置 App**，新 App 从自己的仓库经 registry 发布（`App Kit`）。

平台层用 CPP 接缝（Composed Platform Providers）支持企业版：`interfaces.py` 定义 Protocol，核心从不判断跑的是哪个版本。

> **类比 · 大楼和租户**
>
> - **大楼结构和门禁** = 核心：会话、记忆、闸门、治理。
> - **租户** = App：各自装修，但进出都走大楼门禁。
> - **物业规章** = 治理上限：租户可以把自己的门锁得更紧，不能拆大楼的门。
>
> 如果某个界面只能盖进大楼结构里，原则说这是平台的缺陷，不是破例的理由。

端到端：一条 Dashboard 消息

▶ 用户在 Dashboard 发送一条消息 1 / 9



1. 进入后端

消息进入 `_run_chat`，这是聊天运行的核心，由 `chat_turn` 的 owner 模块组合。

2. 消息 hook

`on_message` 可以自动回复、改写、注入或拒绝。

3. 取会话

`get_or_create` 按 session key 拿到会话。warm pool 开启时可以省掉冷启动。

4. 拼上下文

`ContextBuilder` 组装记忆、技能和教训。

5. 发送

`AcpClient` 通过 JSON-RPC 把 prompt 发给 harness 子进程。

6. harness 运行

harness 调 LLM，流式返回文本和工具调用事件。

7. 工具闸门

每个工具调用都由 `on_tool_call` 判定，结果写入 SEL。需要审批时生成卡片，超时就拒绝并说明原因。

8. 回传

事件转成界面更新，流式推给前端。

9. 落盘

追加到 ConversationLog，异步触发记忆整理。`finally` 里 `release()` 会话。

步骤的顺序来自 `Message flow`，具体函数我核对了主要节点，没有逐行跟完 `_run_chat`。

开源许可证

部分	许可证	证据
Kiro Crew	Apache-2.0，Copyright Amazon	`LICENSE`、`NOTICE`
OpenDyslexic 字体	SIL OFL 1.1	`NOTICE`
桌面版的 FFmpeg	LGPL 2.1+，独立可执行文件	`NOTICE`
其他第三方	各自许可证	`THIRD-PARTY-NOTICES`
`kiro-cli`	外部，不在此仓库	请看其自身条款

几处值得注意：macOS 的后台点击路径用了未公开接口（`NOTICE`）；工具标题分类器重写了 Codex 的规则集，保留了 Apache 声明。

场景	允许吗	要做什么
自用	允许	注意默认有匿名遥测，可关闭（`README`）
提 PR	允许	我没有在 CONTRIBUTING 里找到 CLA/DCO 要求
公开 fork	允许	保留 LICENSE、NOTICE；不要用 Kiro 和 Amazon 商标
发安装包	允许	带上 THIRD-PARTY-NOTICES；FFmpeg 要满足 LGPL
对外提供服务	允许	Apache-2.0 无网络条款，但 harness 的条款要另看
上架 App	允许	走 `发布指南`

历史：GitHub API 显示 `LICENSE` 只有 2026-07-06 一次提交（#5），之后未变。

> **不是法律意见**
>
> 这是工程师读代码时的理解，真要分发请咨询律师。

贡献与定制

开发环境

```
git clone https://github.com/kirodotdev/KiroCrew.git
cd KiroCrew
make build
source .venv/bin/activate
```

要求 Python 3.12+、Node.js 22.12+，以及一个已登录的 ACP 后端（`CONTRIBUTING`）。

项目规则

- **AGENTS.md 是路由器，不是手册。** 改某个子系统前，先读 `Read before you touch` 表里对应的规格，并在同一 commit 更新。这是全仓库最有用的一张表。
- 认领 issue：在 T1/T2 的 issue 上评论 `/claim`，7 天期限，最多同时 2 个（`Claiming`）。
- 不要写死 model id。
- 不接受新的内置 App。
- 用户可见的决定记录在 `docs/decisions/`，条目不可变，改变主意要写新条目。动手“修”一个看起来奇怪的文案前先查这里。
- 硬规则由 `AUTOSDE.yaml` 定义，Claude 和 Codex 的 PR reviewer 都执行。
- CI 规则见 `ci-and-reviews.md`。

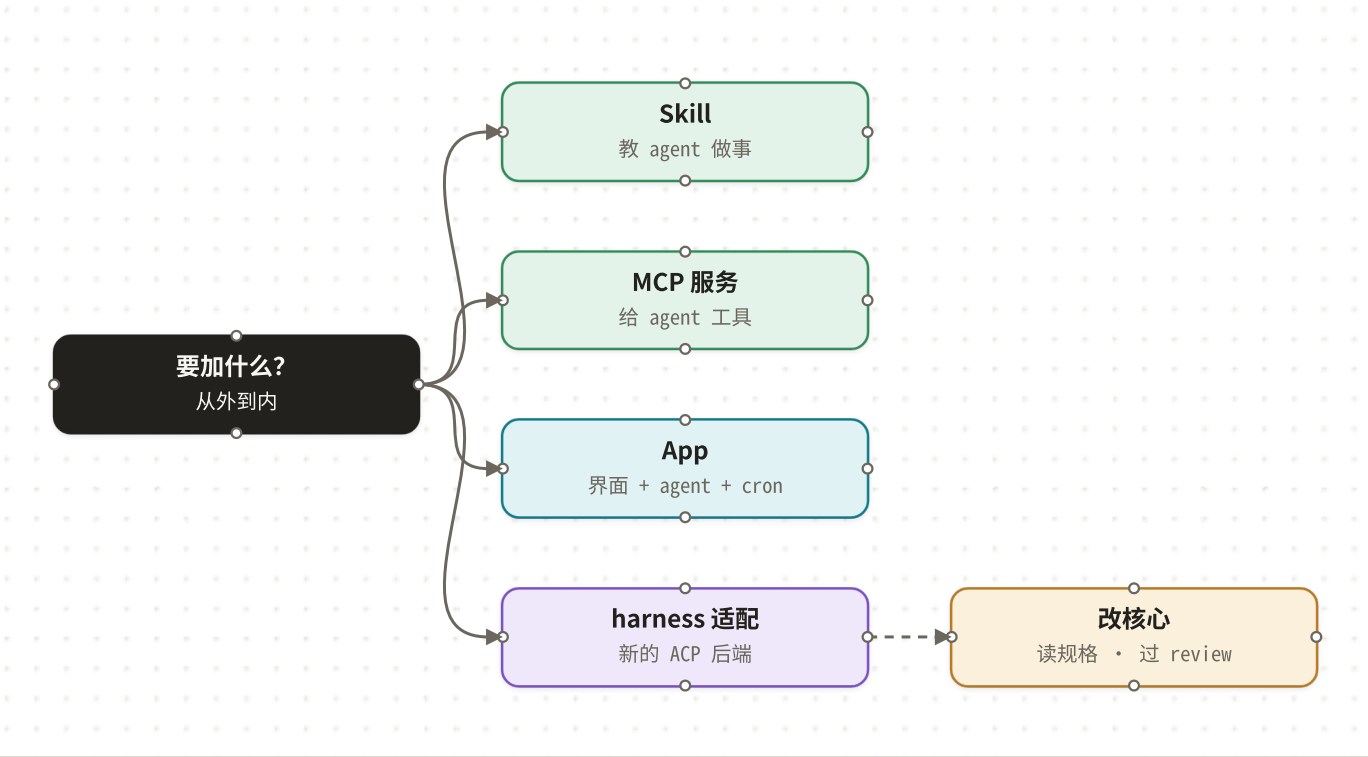
适合入手的方向

- 按 `area:` 标签筛 `tier:T1` 的 issue。
- 写 App 和 Skill，完全不碰核心。
- 文档和规格之间的不一致，比如前面提到的那两处。

需要避开：安全相关的“顺手加固”；第二个 provider（`已明确禁止`）；把内部耦合带回来。

定制路线

想扩展 Kiro Crew



能写成 App 就不要改核心，这是项目的硬性方向。

路线	代价	依赖	适合
Skill	低	Markdown	工作流程、团队习惯
MCP 服务	中	`MCP 规范`	新工具
App	中	`App Kit`	带界面的完整功能
harness 适配	高	要满足 harness parity	接入新 agent
企业版接缝	高	CPP Protocol	内部部署

仓库里没有的东西

- `kiro-cli` **和模型服务**：默认运行时是外部程序，需要单独安装和登录。
- **企业版组件**：通过接缝加载，公开构建没有。
- **内部扫描的标记列表**：`internal-content-scan` 的标记是私有的。
- **遥测接收端、App registry 和下载服务**：在 `download.crew.kiro.dev` 等外部地址。

附录：源码索引

主题	链接
路由表	`AGENTS.md`
架构总览	`overview.md`
模块规格	`system-specs`
入口	`cli.py`、`server.py`
会话	`session.py`
ACP	`client.py`、`client_turn.py`
上下文	`context.py`
记忆	`memory.py`、`vector_memory.py`
闸门	`hooks.py`
治理	`governance.py`
沙箱	`sandbox.py`
审计	`sel.py`
安全深入	`security-deep-dive.md`
前端	`website/AGENTS.md`
原则与治理	`TENETS.md`、`GOVERNANCE.md`